Guía completa del gemelo WMS

smart-wms-twin · PrismaTec-Core · Clean Architecture
Rama feat/wms-prismatec-genesis

Documento de dominio de la infraestructura: estructura, enlaces entre capas, patrones (repository, feature-first), cómo añadir funcionalidades y cómo verificar sin perderse. Pensado para el equipo de backend y para quien deba mantener el gemelo.

No sustituye el código fuente; lo ordena y lo hace navegable.

1. Qué es el gemelo

El gemelo (wms-twin) es un backend en Go que expone el mismo contrato HTTP /api que el backend Flask original (Sentry WMS), para que el admin (React) y el mobile (React Native) puedan apuntar a él sin reescribirse. Por debajo no usa PostgreSQL como fuente de verdad: usa PrismaTec-Core (organismos, ledger, CID, nodo, mesh).

Objetivo de producto: poder conmutar clientes entre Flask y el gemelo (lab → producción cuando la paridad y la operación lo permitan). Flask y el frontend no se modifican en este trabajo.

2. Mapa mental de componentes

```
Clientes (intocados)
  Admin SPA  → cookies sentry_auth + sentry_csrf + X-CSRF-Token
  Mobile RN  → JWT en secure storage + Authorization: Bearer

        ■  /api/*
        ▼
  httpapi (interfaz HTTP Sentry-compatible)
        ■
        ■■ identity / auth
        ■■ application/<feature>   (casos de uso feature-first)
        ■       ■
        ■       ▼
        ■   ports (interfaces Repository)
        ■       ■
        ■       ▼
        ■   infrastructure/persistence + corestore
        ■       ■
        ■■■■■■■■■■■■ PrismaTec-Core Node (organismos, ledger, AIP, mesh)
```

3. Clean Architecture en este repo

Capa	Path	Qué hace
Domain	internal/domain/*	Reglas y tipos puros (p.ej. inventory)
Ports	internal/ports	Interfaces (AdminResourceRepository, Catalog…)
Application	internal/application/<feature>	Casos de uso por feature
Infrastructure	corestore, infrastructure/persistence, mesh	PrismaTec-Core, red, persistencia
Interface	internal/httpapi, cmd/wms-twin	HTTP, main, wiring

Regla de dependencia: las flechas apuntan hacia adentro. httpapi conoce application y ports; application no conoce detalles de HTTP; la persistencia concreta implementa ports.

4. Feature-first: mapa de features

Feature	Paquete Go
Auth / Identity	application/identity + httpapi auth
Catálogo (WH, zones, bins, items)	application/catalog
Receiving / Putaway	application/receiving
Picking	application/picking
Pack / Ship / Fulfillment	application/fulfillment
Sales orders / Purchase orders	application/salesorders, purchaseorders

POS / Dockd / RMA	application/pos, dockd, rma
Ledger de inventario	application/inventoryledger
Admin en migración (mocks BD)	application/admindata → ports.AdminResourceRepository
Integraciones	events, webhooks, inbound, snapshot
Seguridad tokens WMS	security/wmstoken

5. Patrón Repository (pedido Daniel)

Los endpoints aún no migrados no deben mockear JSON en el controlador. El controlador solo autentica, parsea y llama al repositorio. El repositorio persiste organismos PrismaTec-Core como si fueran filas de BD. Los seeds (datos de demostración) viven solo en la capa de datos (admindata.seedIfEmpty) y se eliminan al migrar el dominio real.

```
httpapi.adminGenericCRUD
    → ports.AdminResourceRepository  (interfaz)
        → admindata.Store            (implementación)
            → corestore.Put/Get/List
                → organism.Manager (PrismaTec-Core)
```

Contrato en código: var _ ports.AdminResourceRepository = (*admindata.Store)(nil) garantiza en compilación que el store cumple el port.

6. PrismaTec-Core en el gemelo

Nodo — proceso con identidad y DataDir; arranca en runtime junto al HTTP.

Organismos — cada registro (warehouse, SO, channel…) es un documento versionable con kind + id + CID.

corestore — adaptador WMS sobre organism.Manager (Put/Get/List/Delete).

Ledger — movimientos de stock idempotentes (Adjust/Move/Allocate…) sobre organismos.

AIP — API del nodo Core (puerto aparte); el admin/mobile Sentry no la usan.

Mesh — difusión de eventos entre nodos cuando hay transporte P2P.

7. Auth: por qué admin ≠ mobile

Admin web: cookie HttpOnly sentry_auth (JS no puede leer el JWT) + cookie legible sentry_csrf; las mutaciones envían header X-CSRF-Token. Así se mitiga XSS (no roban el JWT por document.cookie) y CSRF (un sitio tercero no conoce el token CSRF).

Mobile: guarda el JWT en secure storage del dispositivo y envía Authorization: Bearer. No usa cookies de sesión web; no aplica el mismo CSRF de navegador.

El gemelo acepta ambos modos en authenticatedUser / RequireAuth.

8. Arranque y wiring

cmd/wms-twin + internal/runtime construyen el Node Core, el corestore, identity, ledger, cada application service y el Server HTTP. NewWithIdentity en httpapi/server.go es el composition root: aquí se inyectan dependencias concretas.

```
core.NewNode(Config)
  → corestore.New(node.Organisms(), nodeID)
  → inventoryledger.New(store)
  → catalog.New(store, ledger)
```

```
  → receiving / picking / fulfillment / …
  → admindata.New(store)   // port AdminResourceRepository
  → httpapi.Server{...}.Handler()
```

9. Cómo añadir una funcionalidad nueva (paso a paso)

1. Ubica la feature (¿catálogo, floor, admin, integración?).
2. Si es dominio nuevo: tipos en domain/ o en application//.
3. Persistencia: métodos en el repo de la feature o port en internal/ports + implementación sobre corestore.
4. Caso de uso: función en application/ (sin HTTP).
5. HTTP: handler delgado en httpapi (auth, decode JSON, llamar use-case, writeJSON).
6. Registrar ruta en Server.Handler() mux.
7. Tests unitarios del use-case + test HTTP si el contrato es público.
8. Si reemplaza un mock admin: quitar seed en admindata.seedIfEmpty para ese resource.
9. Actualizar docs/CASOS_USO_FRONTEND.md o inventario de endpoints si el frontend lo consume.

10. Dónde tocar qué (cheat sheet)

Quieres…	Archivo / paquete
Nueva ruta /api/...	httpapi/*.go + mux en server.go
Regla de stock	application/inventoryledger + domain/inventory
Nuevo maestro (SKU, bin)	application/catalog
Mock admin estilo BD	application/admindata (NO el handler)
Contrato de repo	internal/ports
Adapter PrismaTec	corestore / infrastructure/persistence
Login / CSRF / cookies	httpapi/auth.go + security/auth
Token integración POS	security/wmstoken
Script de demo equipo	scripts/team-curl-demo.sh
E2E admin cookies	scripts/admin-e2e-cookie.sh
E2E mobile Bearer	scripts/mobile-e2e-bearer.sh

11. Cómo seguir dependencias en el código

Desde una ruta HTTP: localiza el HandleFunc en server.go → método del Server → servicio en application → llamadas a store/ledger. Para un tipo de organismo, busca el kind string (p.ej. wms_admin_channel) en admindata o en el repo de la feature.

```
# En el repo wms-twin
rg -n "HandleFunc\(\"/api/admin" internal/httpapi/
rg -n "wms_admin_channel|Kind\(" internal/application/admindata/
rg -n "func \(s \*Server\) admin" internal/httpapi/
go test ./internal/application/catalog/ -count=1
```

12. Verificación (no romper lo que funciona)

```
./scripts/start-lab.sh
./scripts/team-curl-demo.sh
```

```
./scripts/admin-e2e-cookie.sh
./scripts/mobile-e2e-bearer.sh
go test ./... -count=1
```

Si un cambio rompe auth dual, floor o listados admin, revertir antes de ampliar features. La paridad de producción con Flask sigue siendo un criterio aparte (E2E UI, dominio profundo, operación).

13. Documentos hermanos en el repo

CLEAN_ARCHITECTURE.md · CAPA_DATOS_MOCKS.md · CASOS_USO_FRONTEND.md · ADMIN_E2E_VERIFICACION.md · PARIDAD_OPERATIVA.md · AUDITORIA_GEMELO.md · NivelDeParidadConBackendOriginal.md

---

Fin de la guía. Mantener este PDF alineado cuando cambien ports, features o el composition root.